CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

NAAC GRADE A+ ACCREDITED UNIVERSITY

2025 QS UNIVERSITY RANKINGS ASIA
#1 AMONG ALL PRIVATE UNIVERSITIES IN INDIA

DATABASE MANAGEMENT SYSTEMS MINI PROJECT REPORT

Vehicle Service Management System

Academic Year: 2026–27

SUBMITED BY:

Name : S S SAHIL UID: 25BCD10109

Branch : BCA (DS) Section: 25BCD-2 Group: B

Semester : 3

SUBMITED TO:

Name : Ms.Harkamal kaur Designation : ASSISTANT PROFESSOR

Sign:_____

TABLE OF CONTENTS

Section	Topic Name	Description
1.0	Introduction	Overview of vehicle service centers and the need for a database.
2.0	Objective & Problem Statement	Problems of manual registers and main project goals.
3.0	System Modules	Detailed breakdown of the 5 main system modules.
4.0	Tools & Technologies Used	Database server, client interface, and SQL language.
5.0	Database Tables & Data Dictionary	Detailed table columns, data types, and primary/foreign keys.
6.0	Database Normalization (Simple Explanation)	Simple 1NF, 2NF, and 3NF explanation in points.
7.0	SQL DDL Code (Table Creation)	Standard CREATE TABLE queries with primary and foreign keys.
8.0	SQL DML Code (Sample Data)	INSERT INTO queries with realistic student-friendly data.
9.0	Important SQL Queries	Practical queries (SELECT, WHERE, JOIN, and GROUP BY).
10.0	Output Screenshots	Printed terminal output screenshots from MySQL CLI.
11.0	Conclusion	Summary of learnings and future improvements.

1.0 INTRODUCTION

1.1 Overview

Vehicles such as cars, motorcycles, and scooters require periodic servicing to remain in good working condition. Common vehicle maintenance jobs include changing engine oil, replacing brake pads, checking batteries, tuning engines, and washing. An automobile service garage or workshop deals with multiple vehicle owners, different vehicle models, mechanics with varied skills, and day-to-day billing.

To manage these activities smoothly, a service center needs a reliable way to store information about owners, their automobiles, which mechanic worked on which job, and how much was paid. This project implements a relational database in SQL to manage vehicle servicing efficiently.

1.2 Need for a Database Management System

Traditionally, many neighborhood garages maintain their records using physical paper notebooks, job cards, or basic spreadsheet files. However, manual record-keeping causes several everyday difficulties:

- **Risk of Missing Records:** Paper notebooks easily get stained with motor oil, torn, or misplaced over time.
- **Slow Record Searching:** Finding the past service history of a vehicle requires turning through dozens of handwritten register pages.
- **Duplicate Work:** Every time a customer visits the garage, their name, phone number, and car details have to be written down again.
- **Calculation Mistakes:** Manual calculation of labor costs and taxes can lead to mathematical mistakes and billing arguments with customers.
- **Lack of Tracking:** Workshop managers cannot easily see how many cars were repaired in a week or check which mechanic completed which job.

1.3 Scope of the Project

The Vehicle Service Management System is designed to provide a straightforward and practical solution for small to medium-sized vehicle service workshops. The scope includes:

- **Customer Records:** Maintaining contact details for vehicle owners.
- **Vehicle Records:** Storing vehicle number, model, brand, and fuel type.
- **Mechanic Profiles:** Recording mechanic names and their repair specializations.
- **Service Tracking:** Logging repair jobs, service dates, costs, and work status.
- **Billing:** Generating bills with service tax and recording payment modes (Cash, UPI, Card).

2.0 OBJECTIVE & PROBLEM STATEMENT

2.1 Problem Statement

In a non-computerized vehicle service garage, managing day-to-day operations manually leads to several practical issues:

- **1. Data Redundancy:** Customer and vehicle details are repeatedly written down in multiple registers, wasting time and causing inconsistent entries.
- **2. Inefficient Job Tracking:** Garage supervisors cannot quickly check which vehicles are currently pending, in progress, or ready for customer pickup.
- **3. Lack of Accountability:** If a repaired vehicle has issues after delivery, it is difficult to identify which mechanic did the work without clear records.
- **4. Unorganized Payment Records:** Tracking total daily income and distinguishing between cash and digital payments is slow and prone to errors.

2.2 Project Objectives

The primary objective of this project is to build a simple, clean, and reliable relational database system using SQL to overcome the limitations of manual record-keeping. Specific objectives include:

- **Centralized Data Storage:** Store all information related to customers, vehicles, mechanics, service records, and bills in a single relational database.
- **Data Integrity:** Use Primary Keys and Foreign Keys to ensure that every vehicle is linked to a valid customer and every service is linked to an assigned mechanic.
- **Eliminate Duplicate Data:** Apply normalization so customer details are stored only once, even if a customer brings multiple vehicles for service.
- **Fast Information Retrieval:** Write simple SQL queries to search vehicle history, list pending service jobs, and calculate billing summaries.
- **Beginner-Friendly Architecture:** Keep the database design clean, simple, and straightforward so it is easy to understand, demonstrate, and defend in college lab exams.

3.0 SYSTEM MODULES

The project is structured into five core modules. Each module handles a distinct task of workshop operations while remaining connected through foreign keys.

3.1 Customer Module

The Customer Module manages the personal information of vehicle owners who bring their automobiles to the service center.

- **Stores Customer Details:** Maintains customer ID, full name, phone number, email address, and city.
- **Quick Lookup:** Allows the garage staff to search for an existing customer using their phone number or customer ID.
- **One-to-Many Relationship:** A single customer can own and service multiple vehicles without repeating their personal contact info.

3.2 Vehicle Module

The Vehicle Module records the technical and registration details of each automobile brought in for maintenance.

- **Stores Vehicle Specifications:** Maintains unique vehicle ID, license plate number, brand (e.g., Maruti, Hyundai, Tata), model name, and fuel type (Petrol, Diesel, EV).
- **Owner Linking:** Uses customer_id as a foreign key to link each vehicle directly to its registered owner.

3.3 Mechanic Module

The Mechanic Module keeps a record of workshop mechanics and their technical skills.

- **Stores Mechanic Details:** Maintains mechanic ID, mechanic full name, specialization (e.g., Engine Expert, Brakes & Suspension, Electrical), and phone number.
- **Job Allocation:** Ensures each repair job is assigned to an appropriate technician based on their technical skills.

3.4 Service Records Module

The Service Records Module tracks individual maintenance and repair visits from start to finish.

- **Records Service Tasks:** Logs service ID, vehicle ID, assigned mechanic ID, service date, service type (e.g., Engine Oil Change, Brake Replacement), and service cost.
- **Status Monitoring:** Tracks the repair progress using clear status values: Pending, In Progress, or Completed.

3.5 Billing Module

The Billing Module handles customer invoicing and payment records once repair work is complete.

- **Bill Calculation:** Records bill ID, service ID, bill date, base service cost, tax amount (18% GST), and total amount payable.
- **Payment Mode:** Stores the payment method used by the customer, such as Cash, UPI (GPay/PhonePe), Debit Card, or Credit Card.

4.0 TOOLS & TECHNOLOGIES USED

This mini project is developed using standard, beginner-friendly tools commonly taught in academic database courses.

Tool / Technology	Type	Usage in Project
MySQL Server 8.0	Relational Database Management System	Acts as the database engine to store tables, enforce constraints, and execute SQL statements.
MySQL Workbench / CLI	Database Client & Interface	Used as the query interface to write SQL scripts, view table data, and execute joins.
SQL (Structured Query Language)	Query Language	Used to create tables (DDL), insert records (DML), and write analytical reports (DQL).
Windows / Linux OS	Operating System	Standard computer environment used for running the MySQL database server.

4.1 Why MySQL is Used

MySQL was chosen for this project for the following reasons:

- **Standard SQL Syntax:** It follows ANSI SQL standards, making commands easy to write, learn, and understand.
- **High Reliability:** It ensures that records are safely saved and consistent using ACID properties.
- **Open Source & Free:** It is completely free to install and widely available across academic computer labs.
- **Multi-Table Join Support:** It allows fast and simple joins across multiple tables to generate consolidated service and billing reports.

5.0 DATABASE TABLES & DATA DICTIONARY

The database contains five relational tables. Below are the field names, data types, constraints, and descriptions for each table.

5.1 Table: Customers

Stores personal contact details of vehicle owners.

Field Name	Data Type	Constraint	Description
customer_id	INT	PRIMARY KEY	Unique ID for each customer.
name	VARCHAR(50)	NOT NULL	Full name of the customer.

phone	VARCHAR(15)	NOT NULL	Mobile phone contact number.
email	VARCHAR(50)	NULL	Email address of customer.
city	VARCHAR(30)	NULL	City of residence.

5.2 Table: Vehicles

Stores technical identity and registration data of customer vehicles.

Field Name	Data Type	Constraint	Description
vehicle_id	INT	PRIMARY KEY	Unique ID for each vehicle.
customer_id	INT	NOT NULL, FOREIGN KEY	Links vehicle to Customers table.
vehicle_number	VARCHAR(20)	NOT NULL	State vehicle license plate number.
brand	VARCHAR(30)	NOT NULL	Manufacturer brand (e.g., Maruti, Hyundai).
model	VARCHAR(30)	NOT NULL	Vehicle model (e.g., Swift, i20).
fuel_type	VARCHAR(15)	NULL	Fuel type: Petrol, Diesel, CNG, or EV.

5.3 Table: Mechanics

Maintains records of workshop mechanics and their technical skills.

Field Name	Data Type	Constraint	Description
mechanic_id	INT	PRIMARY KEY	Unique ID for each mechanic.
mechanic_name	VARCHAR(50)	NOT NULL	Full name of the mechanic.
specialization	VARCHAR(30)	NULL	Skill area (e.g., Engine, Brakes, Electrical).
phone	VARCHAR(15)	NULL	Contact phone number of mechanic.

5.4 Table: Service_Records

Tracks maintenance tasks, scheduled repairs, and workshop statuses.

Field Name	Data Type	Constraint	Description
service_id	INT	PRIMARY KEY	Unique ID for each service record.

vehicle_id	INT	NOT NULL, FOREIGN KEY	Links service to Vehicles table.
mechanic_id	INT	NOT NULL, FOREIGN KEY	Links service to Mechanics table.
service_date	DATE	NOT NULL	Date when service was performed.
service_type	VARCHAR(50)	NOT NULL	Type of repair work.
status	VARCHAR(20)	NOT NULL	Status: Completed, In Progress, Pending.
cost	DECIMAL(10,2)	NOT NULL	Labor/service cost in INR.

5.5 Table: Bills

Maintains bills, tax amounts, and payment methods.

Field Name	Data Type	Constraint	Description
bill_id	INT	PRIMARY KEY	Unique billing invoice number.
service_id	INT	NOT NULL, FOREIGN KEY	Links bill to Service_Records table.
bill_date	DATE	NOT NULL	Date of bill issue.
service_cost	DECIMAL(10,2)	NOT NULL	Base service cost.
tax	DECIMAL(10,2)	NOT NULL	Tax amount (18% GST).
total_amount	DECIMAL(10,2)	NOT NULL	Total payable amount (cost + tax).
payment_mode	VARCHAR(20)	NOT NULL	Mode: Cash, UPI, Debit Card, Credit Card.

6.0 DATABASE NORMALIZATION (SIMPLE EXPLANATION)

Normalization is a step-by-step technique in database design that helps organize data cleanly into tables to avoid duplicate entries and prevent data errors. In this project, all five tables follow the rules up to the Third Normal Form (3NF).

6.1 First Normal Form (1NF)

A table is in 1NF when each column contains only atomic (single) values and there are no repeating groups or lists of values in a single cell.

- **How applied:** We do not store multiple car numbers or multiple phone numbers separated by commas inside a single cell. Each vehicle has its own individual row in the Vehicles table.

6.2 Second Normal Form (2NF)

A table is in 2NF when it is in 1NF and all non-key columns depend entirely on the primary key, with no partial dependencies on part of a composite key.

- **How applied:** Each table in our database uses a single primary key (such as customer_id, vehicle_id, mechanic_id, service_id, bill_id). Since none of the tables use a multi-column composite key, partial dependency cannot happen, and all tables satisfy 2NF.

6.3 Third Normal Form (3NF)

A table is in 3NF when it is in 2NF and there are no transitive dependencies (i.e., a non-key column depending on another non-key column instead of directly on the primary key).

- **How applied:** In an un-normalized table, storing the mechanic's phone number inside the Service_Records table would create a transitive dependency (service_id -> mechanic_id -> mechanic_phone). In our design, mechanic details are kept separately in the Mechanics table, and customer details are kept in the Customers table. This completely eliminates transitive dependencies.

6.4 Normalization Summary

- **Customers Table:** Stores customer profiles independently. Satisfies 3NF.
- **Vehicles Table:** Stores vehicle specifications separately, linked via customer_id. Satisfies 3NF.
- **Mechanics Table:** Stores mechanic personnel data independently. Satisfies 3NF.
- **Service_Records Table:** Links vehicles to mechanics without duplicating personal details. Satisfies 3NF.
- **Bills Table:** Maintains financial invoices linked via service_id. Satisfies 3NF.

7.0 SQL DDL CODE (TABLE CREATION)

Data Definition Language (DDL) is used to create the database and tables with appropriate constraints (PRIMARY KEY and FOREIGN KEY).

```
-- ============================================================
-- VEHICLE SERVICE MANAGEMENT SYSTEM - DDL SCRIPT
-- ============================================================

-- 1. Create and select the database
CREATE DATABASE Vehicle_Service_DB;
USE Vehicle_Service_DB;

-- 2. Create Customers Table
CREATE TABLE Customers (
```

```
    customer_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    phone VARCHAR(15) NOT NULL,
    email VARCHAR(50),
    city VARCHAR(30)
);

-- 3. Create Vehicles Table
CREATE TABLE Vehicles (
    vehicle_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    vehicle_number VARCHAR(20) NOT NULL,
    brand VARCHAR(30) NOT NULL,
    model VARCHAR(30) NOT NULL,
    fuel_type VARCHAR(15),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

-- 4. Create Mechanics Table
CREATE TABLE Mechanics (
    mechanic_id INT PRIMARY KEY,
    mechanic_name VARCHAR(50) NOT NULL,
    specialization VARCHAR(30),
    phone VARCHAR(15)
);

-- 5. Create Service_Records Table
CREATE TABLE Service_Records (
    service_id INT PRIMARY KEY,
    vehicle_id INT NOT NULL,
    mechanic_id INT NOT NULL,
    service_date DATE NOT NULL,
    service_type VARCHAR(50) NOT NULL,
    status VARCHAR(20) NOT NULL,
    cost DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (vehicle_id) REFERENCES Vehicles(vehicle_id),
    FOREIGN KEY (mechanic_id) REFERENCES Mechanics(mechanic_id)
);

-- 6. Create Bills Table
CREATE TABLE Bills (
    bill_id INT PRIMARY KEY,
    service_id INT NOT NULL,
    bill_date DATE NOT NULL,
    service_cost DECIMAL(10,2) NOT NULL,
    tax DECIMAL(10,2) NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    payment_mode VARCHAR(20) NOT NULL,
    FOREIGN KEY (service_id) REFERENCES Service_Records(service_id)
);
```

8.0 SQL DML CODE (SAMPLE DATA INSERTION)

Data Manipulation Language (DML) is used to insert sample test data into the tables so queries can be executed and tested.

```
-- ============================================================
-- VEHICLE SERVICE MANAGEMENT SYSTEM - DML SCRIPT
-- ============================================================

-- 1. Insert Data into Customers Table
INSERT INTO Customers (customer_id, name, phone, email, city) VALUES
(1, 'Rahul Sharma', '9876543210', 'rahul@gmail.com', 'Chandigarh'),
(2, 'Priya Singh', '9812345678', 'priya@gmail.com', 'Mohali'),
(3, 'Amit Verma', '9855512345', 'amit.v@yahoo.com', 'Panchkula'),
(4, 'Sneha Patel', '9988776655', 'sneha@gmail.com', 'Chandigarh'),
(5, 'Vikas Kumar', '9776655443', 'vikas.k@gmail.com', 'Kharar'),
(6, 'Ananya Roy', '9665544332', 'ananya@outlook.com', 'Mohali');

-- 2. Insert Data into Vehicles Table
INSERT INTO Vehicles (vehicle_id, customer_id, vehicle_number, brand, model,
fuel_type) VALUES
(101, 1, 'CH01-AB-1234', 'Maruti', 'Swift', 'Petrol'),
(102, 1, 'CH01-CC-9900', 'Honda', 'Activa 6G', 'Petrol'),
(103, 2, 'PB65-XY-4567', 'Hyundai', 'i20', 'Diesel'),
(104, 3, 'HR03-MN-7890', 'Tata', 'Nexon', 'EV'),
(105, 4, 'CH01-CD-3456', 'Kia', 'Seltos', 'Petrol'),
(106, 5, 'PB11-BK-8822', 'Royal Enfield', 'Classic 350', 'Petrol'),
(107, 6, 'PB65-ZZ-1100', 'Toyota', 'Glanza', 'Petrol');

-- 3. Insert Data into Mechanics Table
INSERT INTO Mechanics (mechanic_id, mechanic_name, specialization, phone) VALUES
(201, 'Ramesh Kumar', 'Engine Expert', '9811122233'),
(202, 'Sukhwinder Singh', 'Brakes & Suspension', '9822233344'),
(203, 'Deepak Sharma', 'General Servicing', '9833344455'),
(204, 'Manoj Yadav', 'Electrical & Battery', '9844455566');

-- 4. Insert Data into Service_Records Table
INSERT INTO Service_Records (service_id, vehicle_id, mechanic_id, service_date,
service_type, status, cost) VALUES
(301, 101, 201, '2026-10-01', 'Engine Oil Change', 'Completed', 1500.00),
(302, 102, 203, '2026-10-03', 'General Periodic Service', 'Completed', 800.00),
(303, 103, 202, '2026-10-05', 'Brake Pad Replacement', 'Completed', 2200.00),
(304, 104, 204, '2026-10-08', 'Battery Health Checkup', 'Completed', 1200.00),
(305, 105, 201, '2026-10-10', 'Full Engine Tune-up', 'Completed', 3500.00),
(306, 106, 203, '2026-10-15', 'Chain Lubing & Tuning', 'In Progress', 600.00),
(307, 107, 202, '2026-10-18', 'Suspension Overhaul', 'Pending', 2800.00);

-- 5. Insert Data into Bills Table
INSERT INTO Bills (bill_id, service_id, bill_date, service_cost, tax,
total_amount, payment_mode) VALUES
(401, 301, '2026-10-01', 1500.00, 270.00, 1770.00, 'UPI - GPay'),
(402, 302, '2026-10-03', 800.00, 144.00, 944.00, 'Cash'),
(403, 303, '2026-10-05', 2200.00, 396.00, 2596.00, 'Debit Card'),
(404, 304, '2026-10-08', 1200.00, 216.00, 1416.00, 'UPI - PhonePe'),
(405, 305, '2026-10-10', 3500.00, 630.00, 4130.00, 'Credit Card');
```

9.0 IMPORTANT SQL QUERIES

To test the system, practical queries using SELECT, WHERE, ORDER BY, GROUP BY, and INNER JOIN were executed.

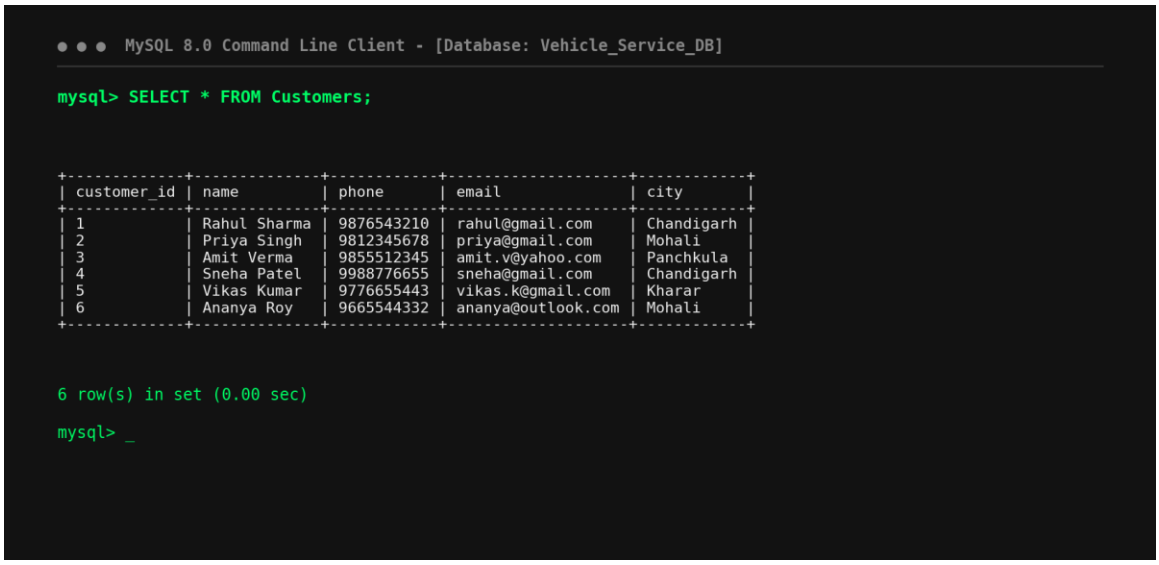
10.0 OUTPUT SCREENSHOTS (MYSQL CLI)

Below are the SQL queries followed by their printed terminal screenshots captured directly from the MySQL command line client.

10.1 Output 1: Display All Customer Records

SQL Statement:

```
SELECT * FROM Customers;
```

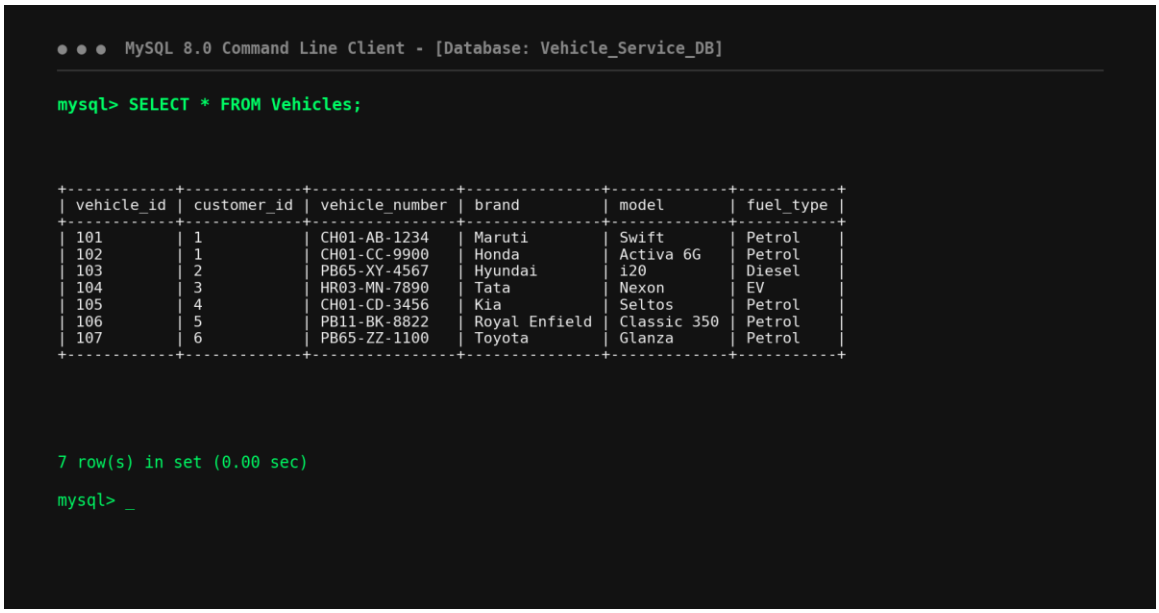


Screenshot 10.1: MySQL CLI output for Customers table

10.2 Output 2: Display All Registered Vehicles

SQL Statement:

```
SELECT * FROM Vehicles;
```

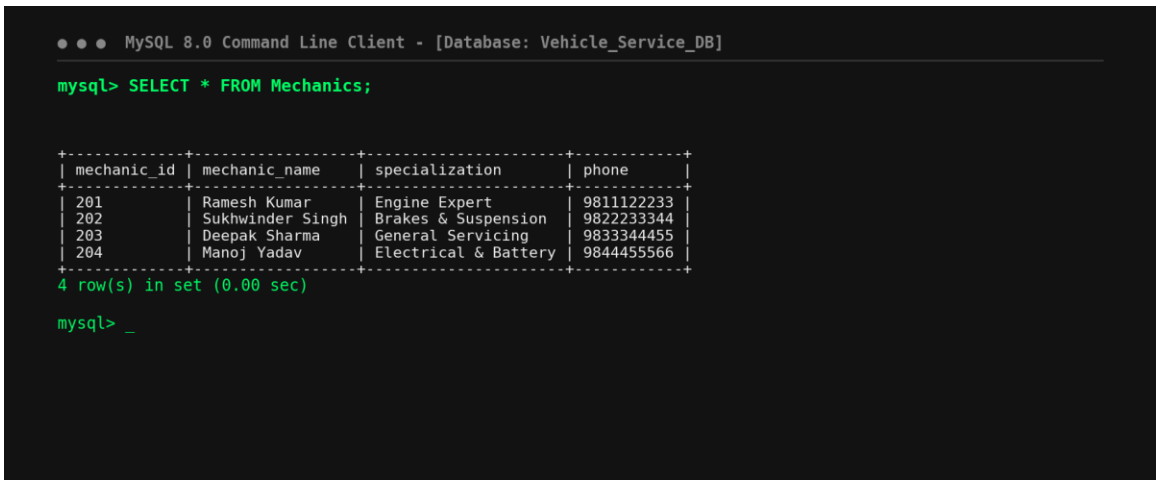


Screenshot 10.2: MySQL CLI output for Vehicles table

10.3 Output 3: Display All Mechanics and Specializations

SQL Statement:

```
SELECT * FROM Mechanics;
```

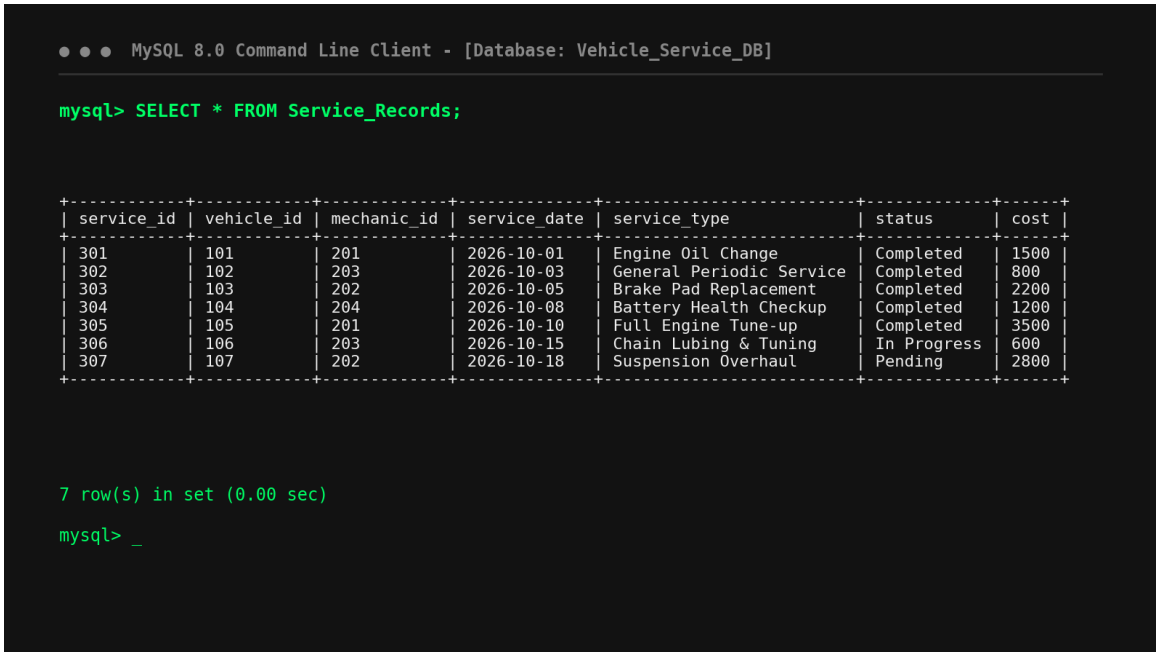


Screenshot 10.3: MySQL CLI output for Mechanics table

10.4 Output 4: Display All Service Records and Status

SQL Statement:

```
SELECT * FROM Service_Records;
```

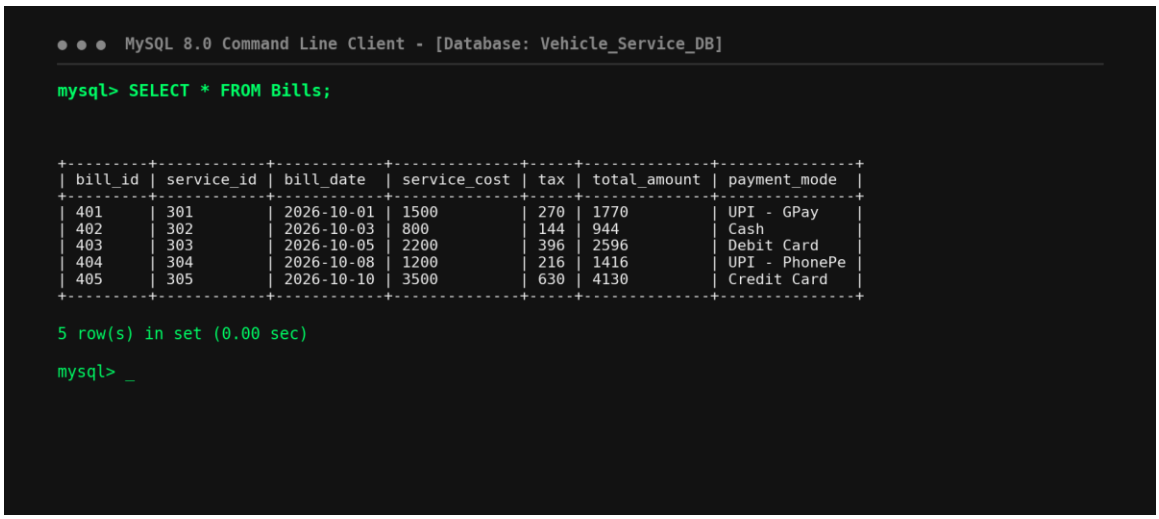


Screenshot 10.4: MySQL CLI output for Service_Records table

10.5 Output 5: Display Invoices and Payment Modes

SQL Statement:

```
SELECT * FROM Bills;
```

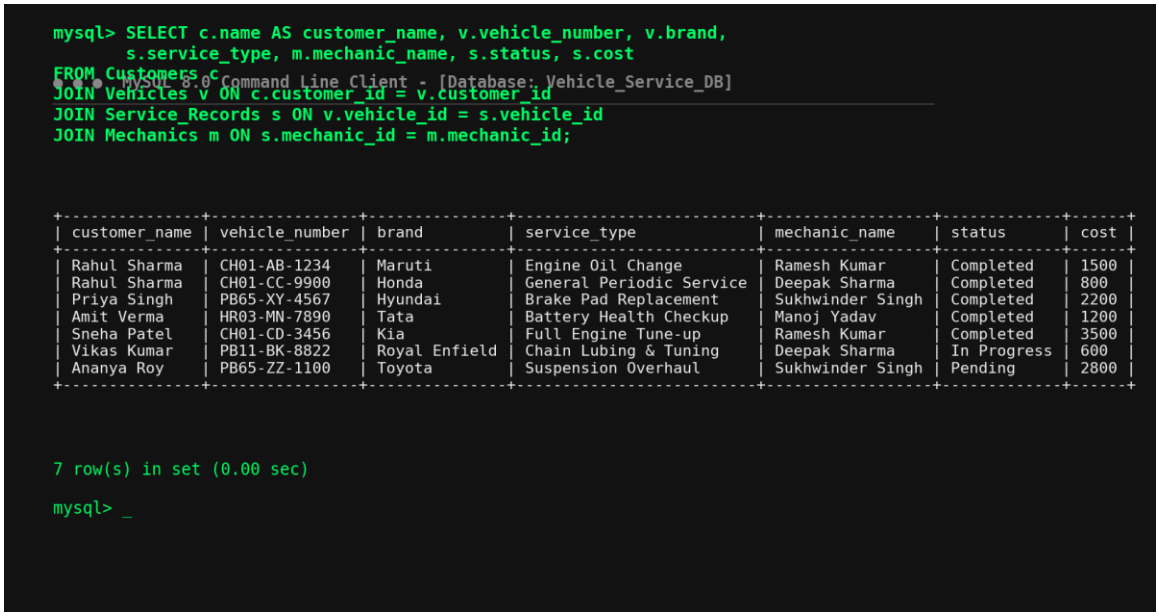


Screenshot 10.5: MySQL CLI output for Bills table

10.6 Output 6: Multi-Table Join for Detailed Service Report

SQL Statement (Joins Customers, Vehicles, Service_Records, and Mechanics):

```
SELECT c.name AS customer_name,
       v.vehicle_number,
       v.brand,
       s.service_type,
       m.mechanic_name,
       s.status,
       s.cost
FROM Customers c
JOIN Vehicles v ON c.customer_id = v.customer_id
JOIN Service_Records s ON v.vehicle_id = s.vehicle_id
JOIN Mechanics m ON s.mechanic_id = m.mechanic_id;
```



Screenshot 10.6: Multi-table join showing combined service and customer information

10.7 Output 7: Aggregation Query (Total Services & Spending per Customer)

SQL Statement (Calculates total services and spending using COUNT and SUM):

```
SELECT c.name,
       COUNT(s.service_id) AS total_services,
       SUM(s.cost) AS total_spent
FROM Customers c
JOIN Vehicles v ON c.customer_id = v.customer_id
```

```
JOIN Service_Records s ON v.vehicle_id = s.vehicle_id
GROUP BY c.name;
```

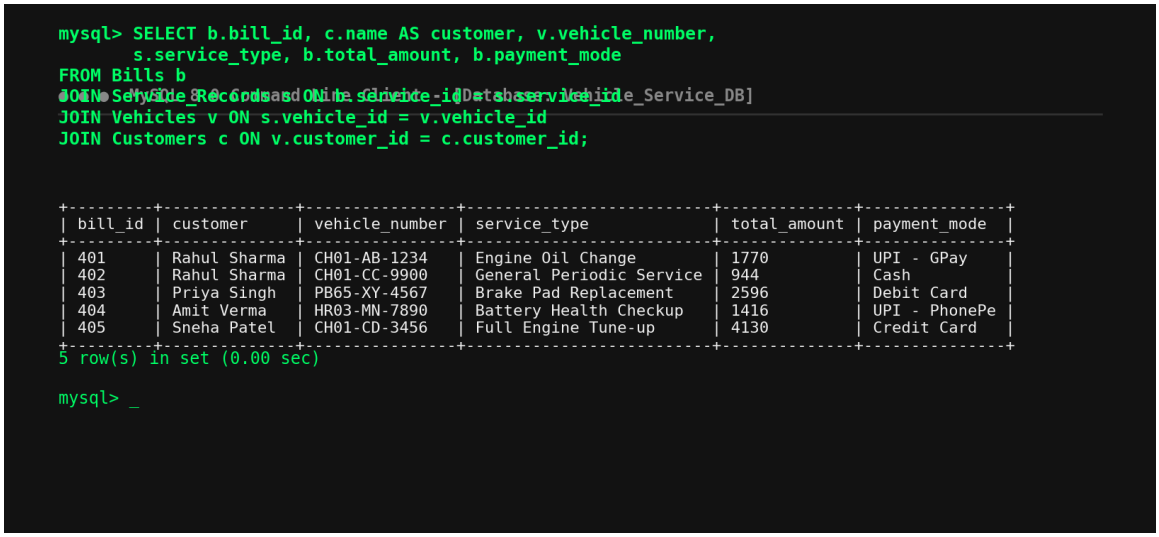


Screenshot 10.7: Aggregate query showing total services and spending per customer

10.8 Output 8: Complete Billing Summary with Payment Modes

SQL Statement (Joins Bills, Service_Records, Vehicles, and Customers):

```
SELECT b.bill_id,
       c.name AS customer,
       v.vehicle_number,
       s.service_type,
       b.total_amount,
       b.payment_mode
FROM Bills b
JOIN Service_Records s ON b.service_id = s.service_id
JOIN Vehicles v ON s.vehicle_id = v.vehicle_id
JOIN Customers c ON v.customer_id = c.customer_id;
```



Screenshot 10.8: Billing report showing total amounts and payment modes

11.0 CONCLUSION

11.1 Project Summary

The Vehicle Service Management System mini project was developed to create a simple, clean, and organized relational database for managing everyday operations in an automobile service center. Using five basic tables—Customers, Vehicles, Mechanics, Service_Records, and Bills—the project successfully demonstrates how relational database concepts solve the problems of manual paper registers.

- **Structured Organization:** Eliminated duplicate data entry by separating customer, vehicle, and mechanic records into distinct tables.
- **Relational Integrity:** Enforced data accuracy using PRIMARY KEY and FOREIGN KEY constraints.
- **Clear Normalization:** Ensured tables satisfy 1NF, 2NF, and 3NF, eliminating update and deletion anomalies.
- **Practical SQL Queries:** Executed single-table queries, multi-table INNER JOINs, and GROUP BY aggregations with printed terminal verification.

11.2 Key Learnings

Through completing this mini project, the following core DBMS concepts were practically understood:

- **1. Schema Design:** How to translate everyday real-world entities into relational tables with appropriate data types.
- **2. Table Linkages:** How foreign keys maintain links between parent and child tables (e.g., connecting a car to its owner).

- **3. Query Writing:** How to write SQL queries to filter, sort, join, and summarize information from multiple tables.
- **4. Database Management:** How relational database systems provide fast, secure, and reliable data storage compared to manual paper logs.

11.3 Future Enhancements

While this mini project fulfills all academic requirements, it can be extended in the future with the following additions:

- **Graphical User Interface:** Connecting the MySQL database to a simple desktop or web frontend (using Python Tkinter, Java, or HTML/PHP) for easier data entry by garage staff.
- **Spare Parts Tracking:** Adding a spare parts inventory table to track parts like filters, spark plugs, and oils separately.
- **Customer SMS Notification:** Adding automatic SMS or email alerts to notify vehicle owners when their servicing is finished.